PIMPRI-CHINCHWAD EDUCATION TRUST'S

Pimpri-Chinchwad College of Engineering

(An Autonomous Institute) • Affiliated to Savitribai Phule Pune University (SPPU) • ISO 21001:2018 Certified by TUV

ASSIGNMENT NO 6

Name:	**Siddhesh Parkar**	**Course:**	Design and Analysis of Algorithm
PRN:	**124B1F021**	**Class:**	TY IT A
Topic:	**0/1 Knapsack with Priorities (Disaster Relief)**	**Batch:**	A1

1. Problem Statement

Disaster Relief Resource Allocation: Following a severe earthquake, a humanitarian agency must transport essential relief supplies (medicines, emergency food, water purification, blankets) to an isolated disaster zone. The delivery truck has a rigid weight capacity of *W* kg. Each of the *N* relief items has a known weight, utility value, and priority rank (3 = Critical, 2 = High, 1 = Normal). Items are indivisible (0/1 choice). Model and solve this problem using the **0/1 Knapsack Dynamic Programming** technique with priority weighting.

2. Course Objectives

- To know the basics of computational complexity of various algorithms.
- To select appropriate algorithm design strategies to solve real-world problems.

3. Course Outcomes

- Analyze the asymptotic performance of algorithms.
- Generate optimal solutions by applying Dynamic Programming strategy.

4. Theory

The 0/1 Knapsack problem is an NP-complete combinatorial optimization problem. When fractional item division is prohibited, greedy approaches fail to guarantee optimality. Dynamic Programming computes optimal sub-allocations by constructing a 2D state table `dp[i][w]` representing the maximum priority-weighted score achievable using a subset of the first *i* items under weight limit *w*.

Composite Priority Scoring & Recurrence:

> **Priority Score Function:** Score(item) = utility + (priority × 100)
>
> **State Transition:**
> dp[i][w] = dp[i-1][w] (if weight[i-1] > w)
> dp[i][w] = max(dp[i-1][w], Score[i-1] + dp[i-1][w - weight[i-1]]) (if weight[i-1] ≤ w)

Comparison of Approaches:

Algorithm	Time Complexity	Optimal Solution?	Pros	Cons
Brute Force	$O(2^N)$	Yes	Simple implementation	Infeasible for large N
Dynamic Programming	O(N × W)	Yes	Guarantees exact optimum	Pseudo-polynomial; requires table memory
Greedy Heuristic	O(N log N)	No	Very fast	Suboptimal for 0/1 indivisible items

5. Algorithm & Pseudo Code

```
procedure Knapsack01(N, W, items):
    Initialize 2D array dp[N+1][W+1] with 0
    for i ← 1 to N do
        score ← items[i-1].utility + (items[i-1].priority * 100)
        for w ← 0 to W do
            dp[i][w] ← dp[i-1][w]
            if items[i-1].weight ≤ w then
                dp[i][w] ← max(dp[i][w], score + dp[i-1][w - items[i-1].weight])
            end if
        end for
    end for
    Traceback through dp table from dp[N][W] to recover selected items
    Sort selected items by priority descending, utility descending
end procedure
```

6. Program Source Code (C++)

```
/*
 * Student Name: Siddhesh Parkar
 * PRN: 124B1F021
 * GitHub Repository: https://github.com/siddhesh160606/DAA_lab
 * Assignment 6: Disaster Relief Resource Allocation Using Dynamic Programming (0/1 Knapsack with Priorities)
 */

#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

struct Item {
    string name;
    int weight;
    int utility;
    int priority;
};

// Returns a score based on utility and priority
int getScore(const Item& item) {
    return item.utility + (item.priority * 100);
}

int main() {
    cout << "Student Name : Siddhesh Parkar\n";
    cout << "PRN          : 124B1F021\n";
    cout << "GitHub Link  : https://github.com/siddhesh160606/DAA_lab\n";
    cout << "------------------------------------------------------\n";

    int N, W;
    cout << "Enter number of items: ";
    cin >> N;

    cout << "Enter truck capacity: ";
    cin >> W;

    vector<Item> items(N);
    cout << "\nEnter item details:\n";
    cout << "Priority: 3 = Critical, 2 = High, 1 = Normal\n\n";

    for (int i = 0; i < N; i++) {
        cout << "Item " << i + 1 << " name: ";
        cin >> items[i].name;

        cout << "Weight: ";
        cin >> items[i].weight;

        cout << "Utility: ";
        cin >> items[i].utility;

        cout << "Priority: ";
        cin >> items[i].priority;
    }

    // DP table
    vector<vector<int>> dp(
        N + 1,
        vector<int>(W + 1, 0)
    );

    // Dynamic Programming
    for (int i = 1; i <= N; i++) {
        int score = getScore(items[i - 1]);
```

```
        for (int w = 0; w <= W; w++) {
            // Don't select the item
            dp[i][w] = dp[i - 1][w];

            // Select the item if it fits
            if (items[i - 1].weight <= w) {
                dp[i][w] = max(
                    dp[i][w],
                    score + dp[i - 1][w - items[i - 1].weight]
                );
            }
        }
    }

    // Find selected items
    vector<int> selected;
    int remainingCapacity = W;
    for (int i = N; i > 0; i--) {
        if (dp[i][remainingCapacity] != dp[i - 1][remainingCapacity]) {
            selected.push_back(i - 1);
            remainingCapacity -= items[i - 1].weight;
        }
    }

    reverse(selected.begin(), selected.end());

    // Sort selected items by priority
    sort(selected.begin(), selected.end(),
        [&](int a, int b) {
            if (items[a].priority != items[b].priority)
                return items[a].priority > items[b].priority;
            return items[a].utility > items[b].utility;
        });

    // Display result
    cout << "\n===================================\n";
    cout << " PRIORITY-WISE RELIEF ALLOCATION\n";
    cout << "===================================\n";

    int totalWeight = 0;
    int totalUtility = 0;

    for (int index : selected) {
        totalWeight += items[index].weight;
        totalUtility += items[index].utility;

        cout << "\nItem: " << items[index].name;
        cout << "\nWeight: " << items[index].weight << " kg";
        cout << "\nUtility: " << items[index].utility;
        cout << "\nPriority: " << items[index].priority;
        cout << "\n";
    }

    cout << "\n-----------------------------------\n";
    cout << "Total Weight : " << totalWeight << " kg\n";
    cout << "Total Utility : " << totalUtility << "\n";
    cout << "Capacity : " << W << " kg\n";

    cout << "\nTime Complexity : O(N * W)";
    cout << "\nSpace Complexity: O(N * W)\n";

    return 0;
}
```

7. Sample Execution Output

```
=========================================================
Student Name: Siddhesh Parkar
PRN         : 124B1F021
GitHub Repo : https://github.com/siddhesh160606/DAA_lab
Assignment  : 6 - Disaster Relief Resource Allocation Using Dynamic Programming (0/1 Knapsack)
=========================================================

Enter number of items: 8
Enter truck capacity: 150

Enter item details:
Priority: 3 = Critical, 2 = High, 1 = Normal

Item 1 name: Medical_Kits
Weight: 25
Utility: 95
Priority: 3

Item 2 name: Water_Purifiers
Weight: 30
Utility: 90
Priority: 3

Item 3 name: High_Calorie_Food
Weight: 40
Utility: 85
Priority: 3

Item 4 name: Blood_Bags
Weight: 15
Utility: 98
Priority: 3

Item 5 name: Thermal_Blankets
Weight: 35
Utility: 70
Priority: 2

Item 6 name: Portable_Generator
Weight: 50
Utility: 80
Priority: 2

Item 7 name: Emergency_Tents
Weight: 45
Utility: 75
Priority: 2

Item 8 name: Hygiene_Kits
Weight: 20
Utility: 50
Priority: 1

====================================
 PRIORITY-WISE RELIEF ALLOCATION
====================================

Item: Blood_Bags
Weight: 15 kg
Utility: 98
```

```
Priority: 3

Item: Medical_Kits
Weight: 25 kg
Utility: 95
Priority: 3

Item: Water_Purifiers
Weight: 30 kg
Utility: 90
Priority: 3

Item: High_Calorie_Food
Weight: 40 kg
Utility: 85
Priority: 3

Item: Thermal_Blankets
Weight: 35 kg
Utility: 70
Priority: 2

-----------------------------------
Total Weight : 145 kg
Total Utility : 438
Capacity : 150 kg

Time Complexity : O(N * W)
Space Complexity: O(N * W)
```

8. Conclusion

The 0/1 Knapsack Dynamic Programming algorithm provides an exact, robust decision-making framework for critical disaster relief logistics. By incorporating priority weighting into the optimization objective, life-saving critical medical assets are selected before general provisions, maximizing life-saving impact within the truck's strict payload capacity in **O(N × W)** time.

GitHub Repository Link

> **Repository Link:** https://github.com/siddhesh160606/DAA_lab